Gestion D'un Jeu De Machine à Bonbons

Elaboré par :

Ghassen Assila

Aladin Zbedi

Ahmed Ammar

Classe :

1AINFO1

Année : 2025/2026

1. Introduction

Ce projet consiste à développer un jeu de machine à bonbons virtuelle. L'objectif est de manipuler des structures de données en langage C, de gérer des tableaux et des listes chaînées, ainsi que d'assurer la persistance des données via des fichiers. Le jeu simule trois distributeurs qui éjectent des bonbons de manière aléatoire pour former des combinaisons rapportant des points.

2. Structures du projet

- **Environnement utilisé :** Code::Blocks en mode console.
- **Organisation des fichiers :**
  - **Main.c :** boucle principale et menu.
  - **Structures.h :** définition des types (Bonbon, Distributeur, Joueur, etc.).
  - **Fonctions statiques.c/.h** : fonctions CRUD utilisant des tableaux.
  - **Fonctions dynamiques.c/.h** : fonctions adaptées aux listes chaînées.

  Gestion_fichiers.c/.h : fonctions de sauvegarde et d'extraction.

3. Conception

La conception repose sur un diagramme de classes UML structuré autour de l'entité centrale Jeu qui gère les tours et les scores. Les entités principales sont :

- **Distributeur :** contient une réserve de bonbons.
- **Bonbon :** caractérisé par une couleur et une valeur.
- **Joueur / Score :** permet de suivre les performances et l'historique des parties

4. Version statique

Cette version utilise des tableaux de taille fixe pour le stockage des données.

4.1. Les structures créées

- **Structure Bonbon** : Elle décrit un bonbon avec sa couleur (chaine de caractère) et sa valeur (entier).

```
typedef struct Bonbon {
    char couleur[TAILLE_COULEUR];
    char type[TAILLE_TYPE];
    int id;
    int valeur;
    struct Bonbon *suivant;
} Bonbon;
```

- **Structure Distributeur :** Elle contient un identifiant, un tableau de bonbons (la réserve et le nombre actuel de bonbons.

```
typedef struct {
    int id;
    Bonbon *premier;
} Distributeur;
```

- **Structure Combinaison :** Elle définit les gains associés à un trio de couleurs spécifique

```
typedef struct combinaison {
    int id;
    char couleurs[MAX_ITEMS][10];
    char types[MAX_ITEMS][10];
    int points;
} combinaison;
```

- **Structure Joueur :** Elle représente l'utilisateur qui participe au jeu

```c
typedef struct {
    char pseudo[20];
    int id;
    int score;
    int meilleur_score;
    int nb_parties;
} joueur;
```

- **Structure score :** Elle est utilisée pour l'archivage et le classement des performances

```c
typedef struct {
    char pseudo[30];
    int points;
    char date[20];
} Score;
```

4.2. Les fonctions créées

4.2.1. Des fonctions de saisie :

Ces fonctions permettent de capturer les données via le clavier pour initialiser les différentes entités du jeu avant leur manipulation.

- **Prototype : Bonbon creerBonbon()**
  Cette fonction interagit avec l'utilisateur pour saisir les caractéristiques d'un bonbon. Elle demande la couleur (ex : " Rouge", " Vent") et la valeur en points.
- **Prototype Joueur créerJoueur()**
  Utilisée au début d'une session de Jeu, elle permet de saisir le pseudonyme du joueur. Elle initialise également le score total à zéro par défaut.
- **Prototype : combinaison creerCombinaisom()**
  Permet de configurer les règles du jeu, L'administrateur saisie à trois couleurs et le nombre de points que cette combinaison rapporte.
- **Prototype : distributeur creerDistributeur(int id)**
  Cette fonction initialise un distributeur. Elle prend en paramètre l'identifiant (1, 2 ou 3). Elle demande ensuite à l'utilisateur combien de bonbons il souhaite ajouter initialement dans la réserve et appelle creerBonbon() en boucle pour remplir le tableau statique.
- **Prototype :** Score creerScore(char pseudo [], int valeur)

Cette fonction est appelée automatiquement à la fin d'une partie, Elle ne nécessite pas forcément de saisie clavier directe car elle récupére le pseudo du Joueur actuel et la valeur finale calculée durant la partie. Elle peut aussi capturer la date système pour l'archivage.

4.2.2. Des fonctions d'affichage

- **Prototype** : void afficherBonbon(Bonbon b)

Affiche proprement sur la console les détails d'un bonbon.

- **Prototype :** void afficherDistributeur(Distribateur d)

Affiche l'ID du distributeur et le nombre de bonbons restants, Elle appelle souvent afficher_des bonbons pour lister le contenu de sa réserve

- **Prototype :** void afficher des_bonbons(Bonbon t[], int n)

C'est la fonction d'affichage de tableau. Elle parcourt le tableau t de taille n et appelle afficheBonbon pour chaque élément. C'est crucial pour visualiser l'état des stocks.

- **Prototype :** void afficherScores(Score t[], int n)

Parcourt le tableau des scores archivés pour afficher le tableau d'honneur {Leaderboard) avec les noms, les scores et les dates.

- **Prototype :** void afficherCombinaison(Combinaison c)
  Cette fonction permet d'afficher une régle de gain specifique. Elle présente à l'écran l'identifiant de la combinaison, les trois couleurs nécessaires pour l'obtenir le gain en points associé.

4.2.3. Fonctions d'affichage d'un tableau

Ces fonctions sont essentielles pour traiter les collections de données, Elles utilisent des boucles (généralement for) pour parcourir les tableaux statiques et présentent les informations de manière organisée à l'utilisateur

- **Prototype** : void afficher des_bonbons(Bonbon t[], int n)
  Cette fonction parcourt un tableau de bonbons t de taille n. Pour chaque élément, elle appelle la fonction un unitaire afficherBonbon est principalement utilisée pour lister le contenu de la réserve d'un distributeur

- **Prototype** : void afficher des combinaisons (Combinaison t[], int n)
  Elle permet d'afficher l'intégralité du tableau des régles de gains. Cela permet au joueur de voir toutes les combinaisons possibles (ex : 3 bonbons de même couleur) et les points correspondant avant de lancer la machine.
- **Prototype :** void afficher des _scores (Score t [], int n)
  Cette fonction affiche le tableau d'honneur (Leaderboard), Elle parcourt la liste des scores archivés pour afficher le nom de chaque Joueur, son score final et la date de sa performance
  4.prototype : void afficher_des_joueurs(Joueur t[], int n)
  Elle est utilisée pour afficher la liste de tous les joueurs enregistrés dans le système, Elle permet de suivre l'évolution des scores totaux cumulés par chaque utilisateur
- **Prototype :** void afficher distributeur (Distributeur d)

Bien que cette fonction affiche un objet "Distributeur", elle est classée ici car elle gére l'affichage du tableau interne (la reserve de bonbons) du distributeur, Elle affiche d'abord l'ID de la machine, puis appelle afficher des bonbons pour lister tout ce que le distributeur contient encore.

4.2.4. Des fonctions d'ajout

- **Prototype :** int ajouterBonbon(Distributeur *d, Bonbon b)

Cette fonction ajout la bonbon b dans le tableau réserve du distributeur d. Elle vérifie d'abord si la capacité maximale (MAX_ BONBONS) n'est pas atteinte, Si l'ajout est possible, le bonbon est placé à l'indice d->nb bonbons, puis ce compteur est incrémenté. Elle retourne en cas de succès ct 0 sinon

- **Prototype :** void ajouterDistributeur(Distributeur tab [], int *n, Distributeur d)

Utilisée lors de l'initialisation de la machine, cette fonction insère un distributeur complet d dans le tableau principal des machines, Le paramètre n (nombre de distributeurs) est passé par adresse pour Jour après l'insertion.

- **Prototype** : void ajouterCombinaison(Combinaison tab [], int *n, Combinaison c)

Permet d'enregistrer une nouvelle régle de gain dans le tableau des combinaisons. Cette fonction est appelée soit au démarrage pour charger les règles par défaut, soit par l'administrateur pour ajouter de nouveaux types de gains.

- **Prototype :** void ajouterJoueur(Joueur tab [], int *n, Joueur j)

Cette fonction enregistre un nouveau profil de joueur dans le tableau des joueurs actifs de la session. Elle permet de garder une trace de tous œux qui ont interagi avec la machine depuis le lancement du programme.

- **Prototype** : void ajouterScore(Score tab [], int *n, Score s)

Appelée à chaque fin de partie, cette fonction insére la performance finale s (pseudo, score, date) dans le tableau historique. C'est tableau qui sera ensuite utilisé pour l'affichage du classement et sauvegardé dans les fichiers.

4.2.5. Des fonctions de suppression

- **Prototype :** Bonbon supprimerBonborn(Distributeur *d)

Cette fonction est appelée lorsqu'un distributeur éjecte un bonbon durant un tour de jeu. Elle retire le bonbon situé l'indice position dans la réserve, Elle effectue un décalage des éléments suivants vers la gauche et décrémente nb_bonbons. Elle retourne le bonbon supprimé pour permettre au programme de calculer le gain associé.

- **Prototype :** void supprimerCombinaison(Combinaison tab [], int *n, int id)

Utilisée pour modifier les règles du jeu, cette fonction retire une combinaison spécifique du tableau des gains en utilisant son identifiant, Elle garantit que le tableau reste compact pour les futures recherches de combinaisons gagnantes.

- **Prototype :** int supprimer Joueur (Joueur tab [], int *n, char pseudo [])

Elle permet de supprimer un profil de joueur, La fonction parcourt le tableau pour trouver le pseudo correspondant. Si le joueur est trouvé, ses données sont écrasées par le décalage des éléments suivants et la taille globale *n est décrémentée.

- **Prtotype** : void supprimerDistributeur(Distributeur tab [], int *n, int id)

Cette fonction permet de retirer un distributeur entier de la machine (par exemple, si l'on passe de 3 à 2 distributeurs), Elle supprime l'élément correspondant dans le tableau des distributeurs et ajuste la taille n.

- **Prototype** : void supprimerScore(Score tab [], int “n, int indice) Elle permet de gérer l'historique en supprimant un enregistrement de score spécifique (par exemple le plus ancien ou un score erroné) à une position donnée dans le tableau des archives.

4.2.5.1. Fonctions d'affichage d'un tableau :

Elles permettent de : parcourir de manière séquentielle les structures de données collectives (tableaux statiques) afin de restituer à l'utilisateur une vue globale et organisée des informations du jeu, Elles servent de pont entre la logique de stockage (tableau) et l'interface utilisateur (console) en transformant des données brutes en listes lisibles

4.2.5.2. Affichage des éléments d'un tableau :

Pour chaque type de donnée, une fonction de parcours a été implémentée :

- **<u>Affichage de la réserve de bonbons :</u>**
  Prototype : void afficher_des_bonbons{Bonbon t {], int n)

Cette fonction parcourt le tableau t contenant n bonbons (généralement la réserve d'un distributeur), À chaque itération, elle appelle la fonction afficherBonbon pour présenter la couleur et la valeur de chaque unité.

- **<u>Affichage des distributeurs (machines) :</u>**

  Prototype : void afficher_distributeur(Distributeur d)

Bien qu'elle traite une structure, elle est essentielle ici car elle liste le contenu du tableau interne au distributeur, Elle affiche le numéro du distributeur affichage de sa réserve à la fonction afficher_des_bonbons.

- **<u>Affichage des règles (combinaison)</u>**

  Prototype : void afficher des combinaisons (Combinaison t [], int n)

Elle liste toutes les combinaisons gagnantes définies dans le système, C'est l’affichage du "tableau des gains” qui permet au joueur de connaitre les points associés à chaque trio de couleurs.

- **<u>Affichage du tableau des scores :</u>**

  Prototype : void afficher_des scores (Score t[], int n)

Cette fonction parcourt le tableau historique des scores, Elle formate l'affichage pour présenter une liste élégante avec le pseudo du joueur, le score atteint et la date, permettant ainsi de visualiser le classement.

- **Des Affichage joueurs enregistrés :**

  Prototype : void afficher des joueurs (Joueur t [], int n)

Elle permet de lister tous les profils de joueurs créés durant la session, affichant leurs noms et leur progression globale (score total cumulé) .

4.3. Exécution du projet

- **Phase d'initialisation :** Le programme commence le générateur de nombres aléatoires srand(time (NULL)). Il crée ensuite 3 distributeurs par défaut via la fonction creerDistributeur et remplit chacun d'eux avec 200 bonbons générés aléatoirement (couleurs et types variés) grâce à remplir_ distributeur_exemple.
- **Phase de navigation :** Une structure do…while affiche le menu principale L'utilisateur interagit en saisissant un entier (sécurisé par lireEntier)
- **Logique de jeu (jouer_partie) :** Le programme demande le nombre de joueurs et de manches. Il crée dynamiquement un groupe de joueurs et demande leurs pseudos.

  À chaque tour, le joueur doit appuyer sur 't'pour simuler le déclenchement de la machine.

La fonction tirer_combinaison_aleatoire retire physiquement des bonbons de distributeurs (utilisation de la fonction de suppression) et calcule les points.

- **Gestion des scores :** À la fin de la partie, les scores sont enregistrés avec la date actuelle, Le menu 3 permet d'afficher les scores, de les tirer (ordre décroissant) et même de supprimer un score spécifique par pseudonyme.
- **Paramétrage** : le menu 2 (menu_parametrer_jeu) offre une gestion complète : modification du nombre de distributeurs, de leur capacité, ou ajout/suppression de combinaisons gagnantes.

Captures d'Écran des tests effectués avec la main :

Capture 1 : Menu Principal et Initialisation

```
===== MACHINE A BONBONS =====
1. Jouer une partie
2. Parametrer le jeu
3. Voir les scores
4. Quitter

Votre choix : |
```

Capture 2 : configuration d'une partie :

```
Votre choix : 1
Donner le nombre de joueurs : 2
Donner le nombre de manches : 2
Donner le pseudo du joueur 1 : nom1
Donner le pseudo du joueur 2 : nom2
```

<u>Capture 3 : Déroulement d'un tirage :</u>

```
===== D®but Manche 1 =====

===== Manche 1 / Tirage 1 =====
(nom1) Appuyez sur 't' pour tirer : t
Combinaison pour nom1 :
Combinaison =
Id : 1
  1) couleur=Bleu | type=epic
  2) couleur=Bleu | type=epic
  3) couleur=Bleu | type=epic
Base score    : 9
Bonus couleurs: 1
Points totaux : 10
Points obtenus : 10
(nom2) Appuyez sur 't' pour tirer : t
Combinaison pour nom2 :
Combinaison =
Id : 2
  1) couleur=Rouge | type=rare
  2) couleur=Rouge | type=rare
  3) couleur=Rouge | type=rare
Base score    : 6
Bonus couleurs: 1
Points totaux : 7
Points obtenus : 7
```

<u>Capture 4 : fin de partie et gagnant :</u>

```
*** Gagnant de la manche 1 : nom1 avec 23 points ***

*** Gagnant final : nom1 avec 23 points ***
Score ajoutÚ avec succÞs.
Score ajoutÚ avec succÞs.
```

5. Version Dynamique

Dans cette version, nous avons remplacé l'intégralité des tableaux statiques par des listes chainées, Cette approche permet une gestion beaucoup plus souple de la mémoire : la machine peut désormais gérer un nombre indéfini de bonbons, de joueurs ou de scores sars saturation arbitraire .

5.1. Les listes chainées

La liste de bonbon :

```c
typedef struct Bonbon {
    char couleur[TAILLE_COULEUR];
    char type[TAILLE_TYPE];
    int id;
    int valeur;
    struct Bonbon *suivant;
} Bonbon;
```

La liste de distributeur :

```c
typedef struct Distributeur {
    int id;
    int capacite;
    int nbBonbons;
    Bonbon *premier;
} Distributeur;
```

```c
typedef struct NoeudDistributeur {
    Distributeur *d;
    struct NoeudDistributeur *prev;
    struct NoeudDistributeur *next;
} NoeudDistributeur;

typedef struct ListeDistributeur {
    NoeudDistributeur *tete;
    NoeudDistributeur *queue;
    int taille;
} ListeDistributeur;
```

La liste de combinaison :

```c
typedef struct combinaison {
    int id;
    char couleurs[MAX_ITEMS][TAILLE_COULEUR_COMB];
    char types[MAX_ITEMS][TAILLE_TYPE_COMB];
    int points;
} combinaison;
```

```c
typedef struct NoeudCombinaison {
    combinaison *c;
    struct NoeudCombinaison *prev;
    struct NoeudCombinaison *next;
} NoeudCombinaison;

typedef struct ListeCombinaison {
    NoeudCombinaison *tete;
    NoeudCombinaison *queue;
    int taille;
} ListeCombinaison;
```

La liste de score :

```c
typedef struct Score {
    char pseudo[TAILLE_PSEUDO];
    int points;
    char date[TAILLE_DATE];
} Score;

typedef struct NoeudScore {
    Score s;
    struct NoeudScore *prev;
    struct NoeudScore *next;
} NoeudScore;
```

```c
typedef struct ListeScore {
    NoeudScore *tete;
    NoeudScore *queue;
    int taille;
} ListeScore;
```

5.2. Fonctions adaptées aux listes chaînées

Le passage aux listes chaînées a nécessité la réécriture (Parcours, Ajout, Suppression)

- **Fonctions de gestion des listes (Expl: Distributeur)**

ListeDistributeur* creerListeDistributeur() : Alloue la structure de contrôle et initialise la tête et la queue à NULL.

void ajouterDistributeurQueue(ListeDistributeur *LD, Distributeur *d) : Ajoute un nouveau distributeur à la fin de la liste, C'est une opération en O(1) grâce au pointeur de queue.

int supprimerDistributeurParId(ListeDistributeur *LD, int id) : Parcourt la liste pour trouver l'élément, réajuste les pointeurs des voisins pour ‘sauter’ le nœud à supprimer puis libère la mémaoire,

- **Fonctions de jeu adaptées :**

Combinaison* tirer_combinaison(ListeDistributeur *LD, int *idc) : Cette fonction parcourt la liste des distributeurs. Pour chaque distributeur, elle appelle retirerBonbonDistributeur qui décroche le premier bonbon de la liste interne du distributeur.

void reindexer(ListeDistributeur *LD) : Une fonction spécifique qui parcourt toute la liste pour remettre à jour les IDs des distributeurs après une suppression.

5.3. Apport personnel

Pour améliorer l'expérience utilisateur et la robustesse du code, j'ai ajouté :

- **Tri dynamique :** La fanction trierScoresDeso(LS) qui réorganise la liste chaînée des scores par ordre décroissant avant l'affichage,
- **Gestion Flexible :** La possibilité de modifier dynamiquement le nombre de distributeur (menu _machine) en ajoutant ou supprimant des noeuds en temps réel.
- **Sécurisation des saisies :** utilisation de LireEntier et LireChaine distributeurs basées sur fgets pour éviter les débordements de tampon

5.4. Utilisation des fichiers

Cette partie assure que les données ne sont pas perdues après la fermeture du programme.

5.4.1. Fonctions d'Extraction

Ces fonctions parsent vos fichiers.txt pour transformer le texte brut en structures de données utilisables par le programme :

- **Extraction Distributeurs :** chargerDistributeurs(LD, FICHIER_DISTRIBUTEURS) extrait l'ID et la capacité de chaque machine stockée
- **Extraction des combinaisons :** chargerCombitaison(LC, FICHIER. COMBINAISON) récupère l'historique des tirages passés.
- **Extraction de saisie :** lireEntier() et lirechaine() extraient les données tapées par l'utlisateur au clavier.

5.4.2. Fonctions Sauvegarde

Cette section décrit comment le programme garantit que les données (score,joueurs et combinaisons) sont conservées après la fermeture du logiciel.

- **Sauvegarde des Scores** : sauvegarderScores(LS, FICHIER_SCORES) enregistre de façon permanente tous les nouveaux scores réalisés.
- **Sauvegarde des Combinaisons :** sauvegarderCombinaisons{LC, FICHIER_ COMBINAISONS) archive toutes les combinaisons qui ont été tirées pendant la session.
- **Sauvegarde des Distributeurs :** sauvegarderDistributeurs(LD, FICHIER_ DISTRIBUTEURS} met à jour l'état des distributeurs (par exemple, si la capacité a été modifiée ou si des bonbons ont été retirés)

5.5. Exécution du projet

Cette section détaille le cycle de vie du programme, de son initialisation à sa fermeture, en suivant le flux d'exécution implémenté dans le fichier main.c

5.5.1. Initialisation Chargement des Données

Dès le lancement, le programme prépare l'environnement de travail, Cette phase cruciale pour la persistance des données :

- **Initialisation de l'aléatoire :** Utilisation de srand(time((NULL)) pour garantir que le remplissage distributeurs et les tirages de combinaisons soient imprévisibles.
- **Chargement des fichiers :** Le programme appelle les fonctions chargerDistributeurs , chargerCombinaison , chargerScores
- **Sécurité et configuration par défaut :** Si aucun distributeur n'est détecté au chargement (nd =0) le code exécute une routine de secours qui crée automatiquement 3 distributeurs par défaut de 200 bonbons chacun, remplis via la fonction remplir_distributeur_exemple

5.5.2. Flux de Contrôle (Menu Principal)

L'exécution repose sur une boucle do...while qui maintient le programme actif jusqu'à ce que l'utilisateur choisisse l'option 4(Quitter). Le pilotage se fait par une structure switch traitant les entrées utilisateur

- **Jouer une partie :** lance le module principal de jeu.
- **Paramétrer le jeu :** accède aux sous-menus pour modifier la machine, les distributeurs ou les combinaisons,
- **Voir les scores :** afficher les scores actuels et une version triée par ordre décroissant
- **Statistiques :** analyse les données de la listé de scores.

5.5.3. Logique Opérationnelle d'une Partie

Lorsqu'une partie est lancé (jouer_partie), l'exécution suit un processus rigoureux :

- **Paramétrage dynamique :** L'utilisateur définit le nombre de joueurs et de manches. Le programme alloue alors dynamiquement une ListeJoueurs.
- **Séquence de tirage :** Chaque manche comporte 3 tirs (TIRS_PAR_MAMNCHE) . Pour chaque joueur, le programme entre dans une boucle d'attente d'entrée clavier :

- L'utilisateur doit appuyer sur t pour déclencher tirer_combinaison .

- Le programme extrait un bonbon de chaque distributeur, calcule les points met à jour le score du joueur instantanément,

- **Gestion des gagnants :** À la fin de chaque manche, le gagnant de la manche est affiché, À la fin de toutes les manches, le Gagnant Final est identifié via trouver_gagnant.
- **Archivage :** Les résultats sont datés(obtenirDateActuelle) et aoutés à la liste globale des scores pour être sauvegarder ultérieurement.

5.5.4. Sortie du Programme et Persistance Finale

L'exécution se termine proprement lorsque l'utilisateur choisit de quitter, Le programme garantit l'intégrité des données par deux étapes finales :

- **Sauvegarder physique :** Les listes en mémoire (Distributeurs, Combinaisons, Scores) sont écrites dans leurs fichiers respectifs (distributeurs.txt,ect) via les fonctions sauvegarder.
- **Nettoyage de la mémoire :** Pour éviter toute fuite de mémoire, le programme libère explicitement chaque liste chaînée grâce aux fonctions libererListe... avant de retourner 0.

<u>Capture d'écran des test effectués avec la main</u>

```
===== MACHINE A BONBONS =====
1. Jouer une partie
2. Parametrer le jeu
3. Voir les scores
4. Quitter
Votre choix : 1
```

```
=== NOUVELLE PARTIE ===
Nombre de joueurs : 2
Nombre de manches : 1
Pseudo joueur 1 : nom1
Pseudo joueur 2 : nom2
```

```
===== MANCHE 1 =====

--- Tir 1 ---
(nom1) Appuyez sur 't' : t
======
Combinaison id = 1
  1) couleur=Bleu     | type=ordinaire -> 1 point(s)
  2) couleur=Bleu     | type=ordinaire -> 1 point(s)
  3) couleur=Bleu     | type=epic      -> 3 point(s)

Regles de calcul des points :
 - ordinaire = 1 point
 - rare      = 2 points
 - epic      = 3 points
 - 3 memes couleurs : +2 points bonus
 - 2 memes couleurs : +1 point bonus

Base score    : 5
Bonus couleurs: 2
Points totaux : 7
===
Points : 7
```

```
(nom2) Appuyez sur 't' : t
======
Combinaison id = 2
  1) couleur=Vert     | type=epic      -> 3 point(s)
  2) couleur=Bleu     | type=rare      -> 2 point(s)
  3) couleur=Rouge    | type=ordinaire -> 1 point(s)

Regles de calcul des points :
 - ordinaire = 1 point
 - rare      = 2 points
 - epic      = 3 points
 - 3 memes couleurs : +2 points bonus
 - 2 memes couleurs : +1 point bonus

Base score    : 6
Bonus couleurs: 0
Points totaux : 6
===
Points : 6
```

```
*** Gagnant manche 1 : nom2 (21 pts) ***

=== GAGNANT FINAL : nom2 (21 pts) ===
```

6. Conclusion

le projet de la Machine à Bonbons a permis de concevoir une application complète en langage C, alliant gestion de structures de données complexes et interactivité utilisateur. Ce travail à abouti à un système robuste capable de simuler une expérience de jeu réaliste tout en assurant une gestion rigoureuse de l'information.

Points clés retenus :

- **Flexibilité** : L'utilisateur des listes chainées permet une gestion dynamique des joueurs ct des distributeurs sans limites de taille fixes.
- **Persistance :** Grâce à la gestion des fichiers texte, les scores et les configurations sont conservés d'une session à l'autre.
- **Modularité :** La séparation du code en modules (bonbon, distributeur, score) facilite la maintenance e l'évolution du programme.

Ouverture :

Le programme pourrait évoluer vers une interface graphique plus immersive ou intégrer un système de gestion de comptes sécurisés avec mots de passe pour les Joueurs.